

Knowledge Based Engineering (KBE)

AE4202

Dr.ir. G. La Rocca
FPP

Dr.-Ing. A. Heidebrecht
FPP

Tutorial 2

Python vs. ParaPy object oriented programming

The ParaPy classes

- `Input()`, `@Input`
- `@Attribute`
-

Class structure



Agenda

- Python classes
- ParaPy classes
- ParaPy GUI

Exercise 2 on Python classes

Exercise 3 on ParaPy classes

Typical Python Class Layout

```
class MyClass(Superclass):                                # inherit from Superclass
```

```
    → foo = 1                                              # class attribute
```

```
    → def bar(self):                                       # methods...
        return self.foo + 1
```

```
    → def quz(self):
        return Box(width=self.bar)
```

```
    → def qux(self, spam):
        print(spam)
```

extra arguments

indentation

Python classes

```
class Wing():
    taper = 0.2
    c_root = 4
    def get_c_tip(self):
        print("I take 10 minutes to return")
        return self.c_root * self.taper
```

Class variables

Class method

```
>>> wing = Wing()
>>> wing.get_c_tip()
"I take 10 minutes to return"
0.8
>>> wing.get_c_tip()
"I take 10 minutes to return"
0.8
>>> wing.taper = 0.3
>>> wing.get_c_tip()
"I take 10 minutes to return"
1.2
```

non-caching

Class initialization method

```
class Wing():
    def __init__(self, taper=0.2, c_root=4):
        self.taper = taper
        self.c_root = c_root
        self.c_tip = self.get_c_tip()

    def get_c_tip(self):
        print("I take 10 minutes to return")
        return self.c_root * self.taper
```

```
>>> wing = Wing()
"I take 10 minutes to return"
>>> wing.c_tip
0.8
>>> wing.c_tip
0.8
>>> wing.taper = 0.3
>>> wing.c_tip
0.8
>>> wing.c_tip = wing.get_c_tip()
"I take 10 minutes to return"
>>> wing.c_tip
1.2
```

caching

non-lazy

no dependency tracking

non-lazy

Typical Python Class Layout

```
class MyClass(Superclass):                                # inherit from Superclass

    → foo = 1                                              # class attribute

    → def bar(self):                                       # methods...
        return self.foo + 1

    → def quz(self):
        return Box(width=self.bar)

    → def qux(self, spam):
        print(spam)
```

Typical ParaPy Class Layout

```
class MyClass(Base, Superclasses):
```

```
# inherit from ParaPy Base
```

→ `foo = Input(1)`

```
# Input
```

→ `@Attribute`

```
# Attribute
```

```
def bar(self):  
    return self.foo + 1
```

→ `@Part`

```
# Part
```

```
def quz(self):  
    return Box(width=self.bar)
```

→ `def qux(self, spam):
 print(spam)`

```
# method
```

decorator

decorator

ParaPy adds dependency tracking, caching and lazy evaluation

Python Class

```
class Wing():
    taper = 0.2
    c_root = 4

    def get_c_tip(self):
        print("I take 10 minutes to return")
        return self.c_root * self.taper
```

```
>>> wing = Wing()
>>> wing.get_c_tip()
"I take 10 minutes to return"
0.8
>>> wing.get_c_tip()
"I take 10 minutes to return"
0.8
>>> wing.taper = 0.3
>>> wing.get_c_tip()
"I take 10 minutes to return"
1.2
```

ParaPy Class

```
class Wing(Base):
    taper = Input(0.2)
    c_root = Input(4)

    @Attribute
    def c_tip(self):
        print("I take 10 minutes to return")
        return self.c_root * self.taper
```

```
>>> wing = Wing()
>>> wing.c_tip
"I take 10 minutes to return"
0.8
>>> wing.c_tip
0.8
>>> wing.taper = 0.3
>>> wing.c_tip
"I take 10 minutes to return"
1.2
```

lazy

caching

lazy

dependency tracking

Input() and @Input: inputs to your ParaPy class object

```
class MyClass(Base):
```

no default

```
    foo = Input()
```

required input

Use *only* if you actually have a good default!

```
    foo = Input(<default>)
```

optional input (simple)

```
@Input
```

optional input (derived)

```
def foo(self):
```

```
    return <expression>
```

different syntax, allows adaptive defaults (e.g. rule of thumb for sensible starting values)

Input examples

```
class Wing(Base):  
    span = Input()  
    AR = Input()
```

```
    c_root = Input(4.0)
```

This is *not* a
good default...
(see next slide)

```
@Input()
```

```
def c_tip(self):  
    return self.span / self.AR / 2
```

← @Input: expressions to compute
default value, but can still be
changed

Input defaults – what to use, and when?

- Benefits:
 - Users don't need to specify all inputs, and still get a complete Part
 - Things that rarely change don't need to be specified every time
- Dangers:
 - "Where does that number come from? I did not specify that!"
 - Accidentally treating input variables like constants
- Advice:
 - Provide defaults for values that rarely change: `n_fuselages = Input(1)`
 - Or for modifiable standard settings: `Mach = Input(0.1) # incompressible by default`
 - Defaults used for testing should be recognizably unrealistic!
 - E.g. default chord = thickness = span = 1m
 - ...but even better not to use defaults this way

Make *very obvious*
that values are not
meant to be realistic

Attribute: embodies engineering rule / is an output

```
→ @Attribute(**kwargs)
    def <name>(self):
        <expression>*
        return <expression>
```

```
class Wing(Base):
    # ...

    @Attribute
    def c_tip(self):
        return self.c_root *
self.taper
```

See Exercise 3 code

Part: composition of child objects

```
→ @Part(**kwargs)
def <name>(self):
    return <Class>(*args,
                    hidden=bool,
                    suppress=bool,
                    quantify=int,
                    pass_down=str,
                    map_down=str,
                    **kwargs)
```

```
class Wing(Base):
```

```
# ...
```

```
@Part
```

```
def airfoil:
```

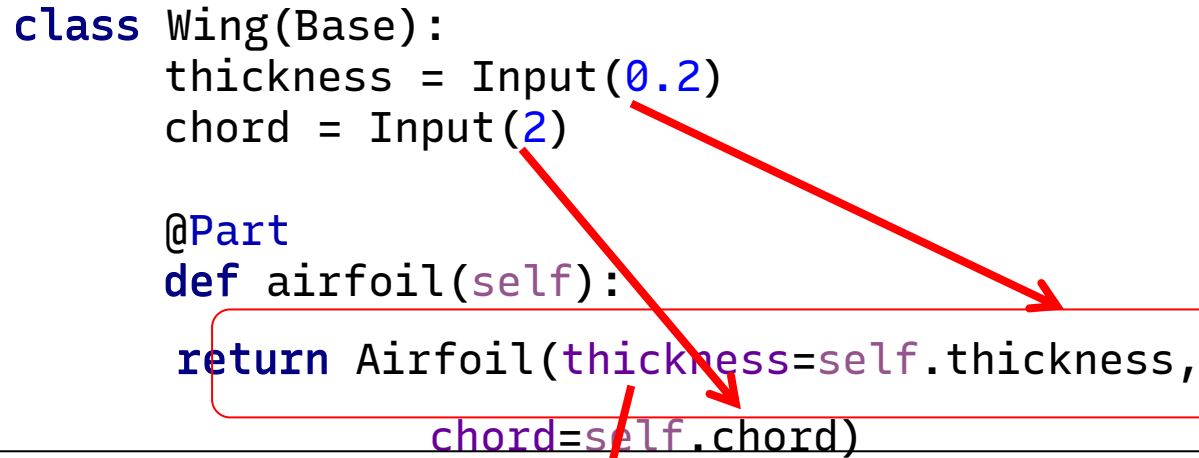
```
    return
```

```
Airfoil(chord=self.c_root)
```

```
class Airfoil(Base):
    chord = Input()
```

Single Part Example

```
class Wing(Base):  
    thickness = Input(0.2)  
    chord = Input(2)  
  
    @Part  
    def airfoil(self):  
        return Airfoil(thickness=self.thickness,  
                        chord=self.chord)
```



```
class Airfoil(Base):  
    thickness = Input()  
    chord = Input()
```

```
return Airfoil(pass_down=["thickness", "chord"],
```

You can use `pass_down` for conciseness

```
>>> wing = Wing()  
>>> wing.airfoil.thickness  
0.2  
>>> wing.airfoil.chord  
2  
>>> wing.airfoil.hidden  
False  
>>> wing.chord = 3  
>>> wing.airfoil.hidden  
True  
>>> wing.chord = 1e-4  
>>> wing.airfoil  
Undefined
```

Quantified Part Example

```
class Airfoil(Base):  
    thickness = Input(1)  
    chord = Input(1)
```

```
class Wing(Base):  
    n_airfoils = Input(2)  
    thickness = Input(0.2)
```

```
@Part  
def airfoils(self):  
    return Airfoil(quantify=self.n_airfoils,  
                   pass_down= ["thickness", "chord"],  
                   chord=0.1*(child.index+1),  
                   label="root_airfoil" if child.index == 0 else "other_airfoil")
```

```
>>> obj = Wing()  
>>> obj.airfoils  
<Sequence root.airfoils at 0x4b4f2e8>  
>>> len(obj.airfoils)  
2  
>>> obj.airfoils[0].chord  
0.1  
>>> obj.airfoils[-1].chord  
0.2  
>>> [item.chord for item in obj.airfoils]  
[0.1, 0.2]
```

See also exercise 3: How is airfoil chord set?

EXE: the geometry-less aircraft

- Aircraft
- Fuselage
- Wing
- Engines & nacelles

EXE 4. See `Aircraft.py` on Brightspace

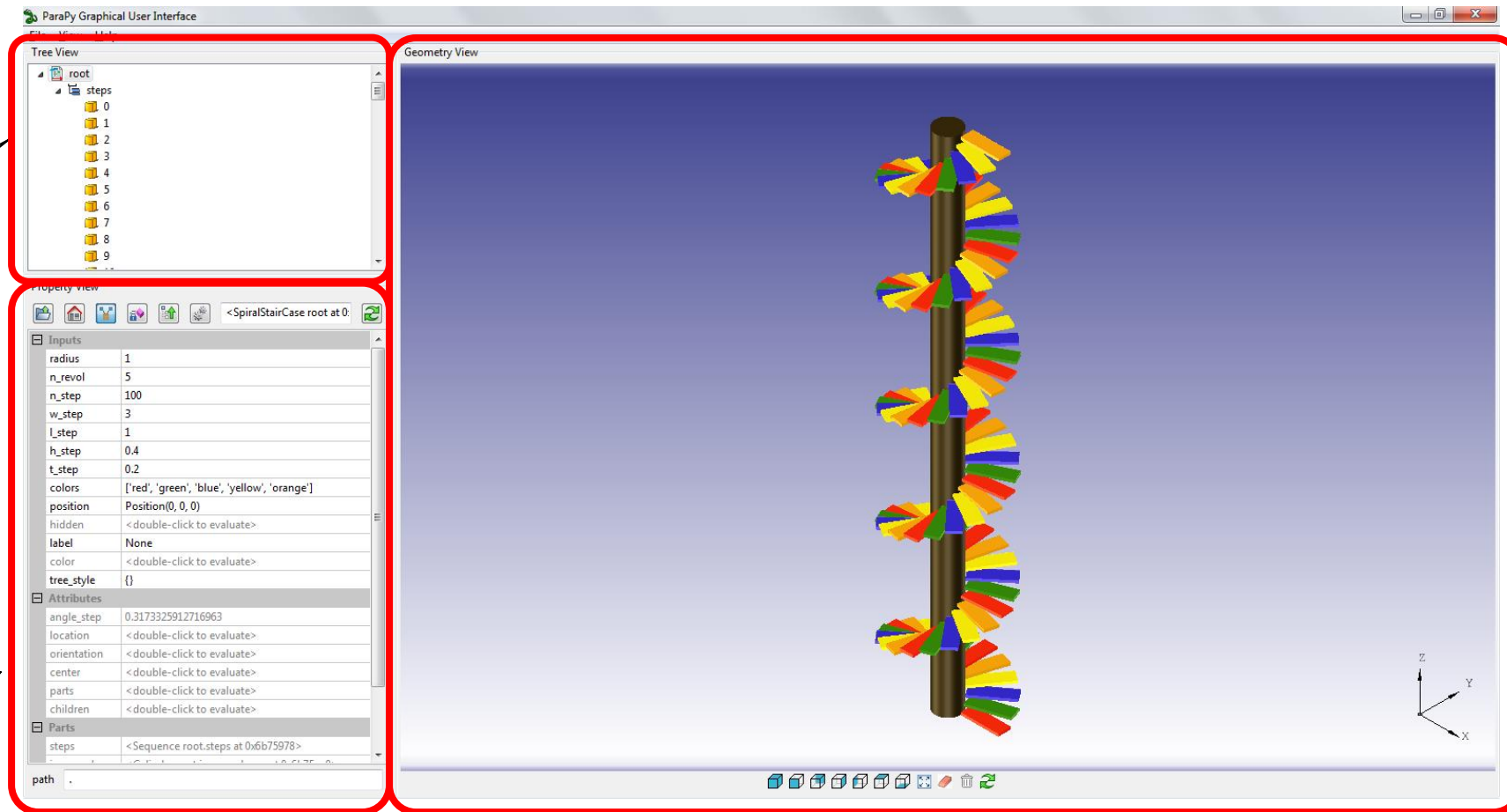
Observe:

- Class structure matches structure shown in parapy GUI
- How does the volume calculation work?

Add:

- Mass calculation (inspired by volume calculation)
- Some additional component, e.g.:
 - interiors in fuselage (use `quantify`)
 - High lift devices in wing
 - ...

ParaPy Graphical User Interface (package: parapy.gui)

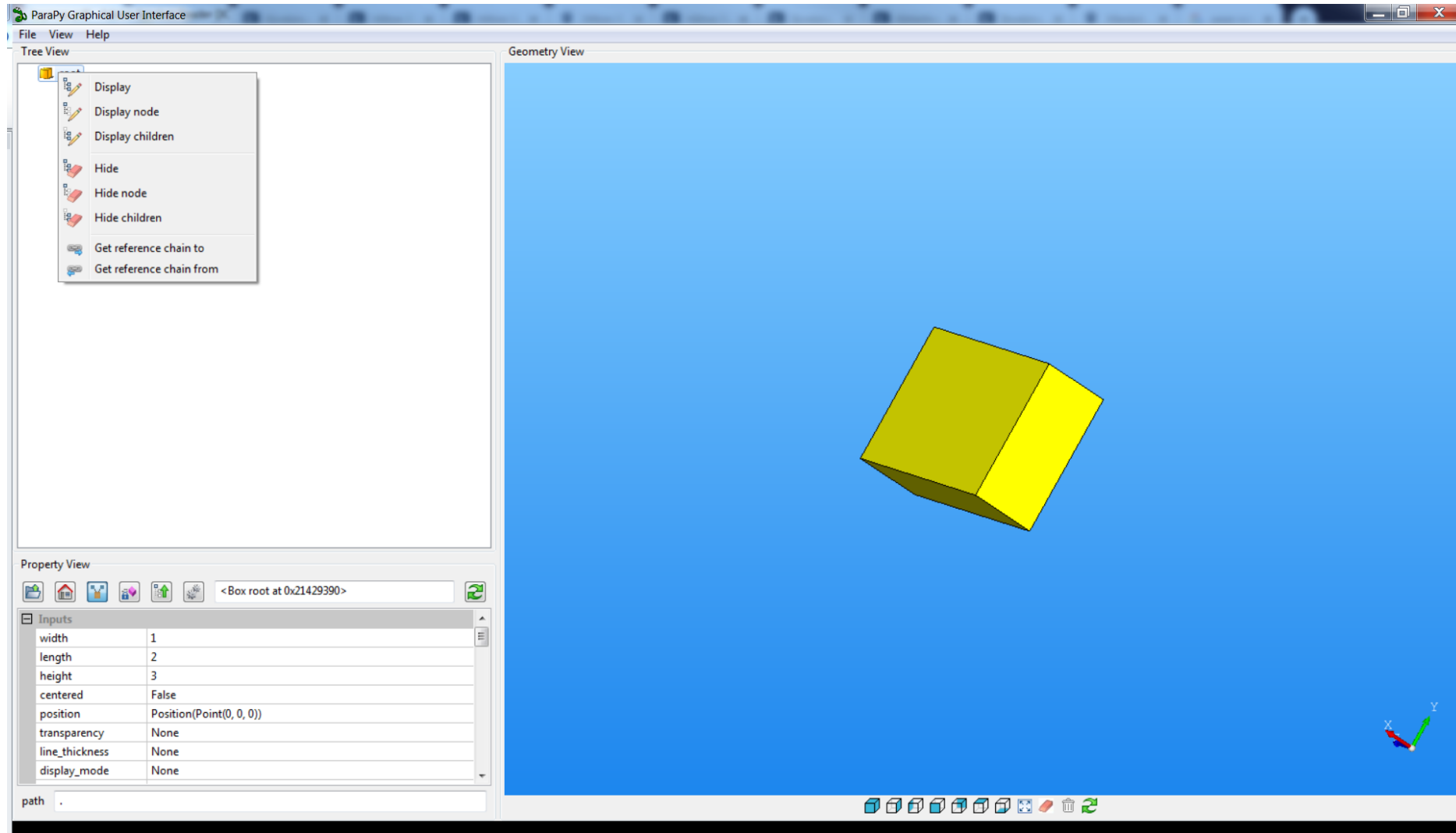


Product tree

Property Grid

Viewport

ParaPy GUI: Product Tree Context Menu



ParaPy GUI: Product Tree Context Menu

Product Tree

left-click		select node
double-click		display all
right-click		context menu
		draw this node and all children
		draw this node only
		draw children only
		hide this node and all children
		hide this node only
		hide children only
		reference chain to other object
		reference chain from other object
		bind object to 'self' (IPython only)

ParaPy GUI: Property Grid

The screenshot shows the 'Property View' window in ParaPy. It features a toolbar with icons for file operations, a tree view on the left, and a main table of properties. The tree view is divided into three sections: 'Inputs', 'Attributes', and 'Parts'. The 'Inputs' section contains properties like 'radius', 'n_rev', 'n_step', and 'steps'. The 'Attributes' section contains 'angle_step', 'location', 'orientation', 'center', 'parts', and 'children'. The 'Parts' section contains 'steps'. The main table displays the values for these properties. Annotations include: 'Level up' pointing to the 'Inputs' section header; 'To root' pointing to the 'Inputs' section header; 'Show inherited slots' pointing to the 'Inputs' section header; 'Show private slots' pointing to the 'Inputs' section header; 'expand / collapse' pointing to the 'Inputs' section header; 'refresh' pointing to the refresh icon in the toolbar; 'object repr' pointing to the '<SpiralStairCase root at 0:' text; 'evaluate all' pointing to the 'evaluate all' icon in the toolbar; 'double-click to evaluate or inspect' pointing to the '<double-click to evaluate>' text in the 'location' row; and 'Path from root' pointing to the 'path' field at the bottom.

Input(), @Input

@Attribute

@Part

Level up

To root

Show inherited slots

Show private slots

expand / collapse

refresh

object repr

evaluate all

double-click to evaluate or inspect

Path from root

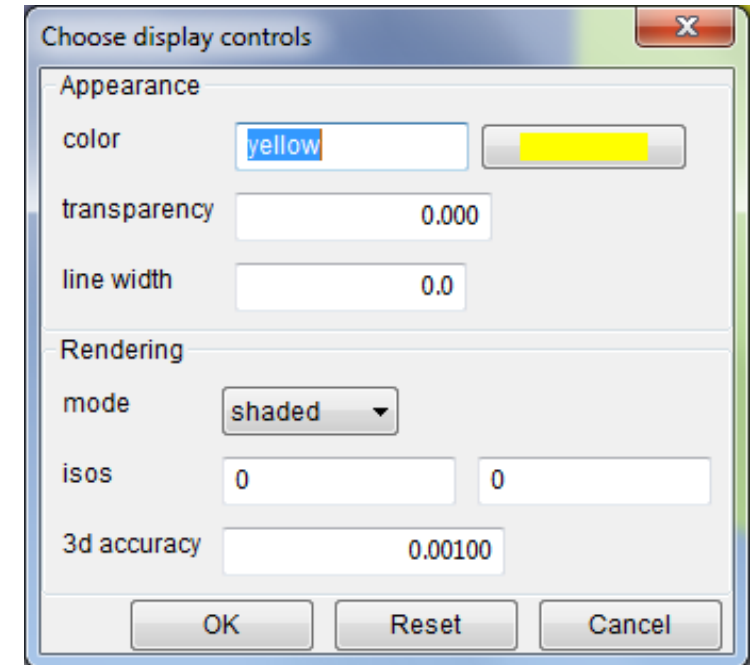
Property	Value
radius	1
n_rev	5
n_step	100
steps	3
colors	['red', 'green', 'blue', 'yellow', 'orange']
position	Position(0, 0, 0)
hidden	<double-click to evaluate>
label	None
color	<double-click to evaluate>
tree style	{}
angle_step	0.3173325912716963
location	<double-click to evaluate>
orientation	<double-click to evaluate>
center	<double-click to evaluate>
parts	<double-click to evaluate>
children	<double-click to evaluate>
steps	<Sequence root.steps at 0x6b75978>

ParaPy GUI: Viewport

f	fit all
c	hide all
w	wireframe
s	shaded
q	hidden-line removal
a	show tessellation
Delete	hide selected objects
left-click ¹	(un-)select object
left-click + motion	rotate
Shift + left-click ¹	(un-)select objects within area
right-click ¹	appearance dialog (with select)
right-click + motion ²	dynamic zoom
Shift + right-click	area zoom
middle-scroll	zoom (x2)
Shift + middle-scroll	fine zoom (x1.2)
middle-click + motion	pan

¹ Hold Ctrl for incremental (un-)select. Object will be added to or removed from previous selection.

² Move the mouse horizontally.



ParaPy GUI: Viewport

